

Instructor Handout 9: Memory Hierarchy and Cache Memory Hierarchy, ROM Family, Mapping Functions, Replacement Algorithms, and Cache Performance

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 18, 2026

Instructor Copy — Not for Student Distribution

9.1 Before You Teach This Lecture

- Stallings, *Computer Organization and Architecture*, 11th ed. (the edition on disk) — this week’s anchor, page-indexed in `supporting_books/Stallings2015/index.md`. Three ranges carry the lecture:
 - **Ch. 4, printed pp. 112–137** — the memory hierarchy: the three characteristics that refuse to cooperate, locality of reference, the hierarchy pyramid (Fig. 4.6, PDF p. 160), and the two-level performance model (§4.3, Fig. 4.8, PDF p. 162).
 - **Ch. 5 §§5.1–5.2, printed pp. 139–142, PDF pp. 187–203** — cache principles and the whole of mapping and replacement. Direct mapping’s rule is Fig. 5.7 (PDF p. 190); the replacement-policy discussion and the hardware implementations (USE bit, index list) are §5.2 (PDF p. 203).
 - **Ch. 6 §6.1, PDF pp. 226–236** — semiconductor main memory: the SRAM/DRAM split and the ROM family.
- **Sourcing warning you must respect at the board.** Stallings 11e covers *only* LRU, FIFO, LFU, and Random. **Optimal (OPT / Belady’s MIN) and Belady’s anomaly appear nowhere in this book.** The deck and Notes therefore present both as standard general treatment with *no page citation*, and you should do the same. If a student asks “which page is Belady’s anomaly on?”, the honest answer is that it is not in our anchor text — it is standard material you would find in an operating-systems text under page replacement. Do not invent a page.
- Patterson & Hennessy, *Computer Organization and Design* (ARM ed.), **Ch. 5 §§5.3–5.4 (p. 397)** — the tag/index/offset framing and AMAT. Page-verified for §5.3 only.
- Hamacher, *Computer Organization and Embedded Systems*, **Ch. 8** — the syllabus’s second anchor for this week. **Chapter-level only** — this chapter is *not* page-indexed for this course, so cite it as “Hamacher Ch. 8” and never with a page number.

- **Handoff from Week 8:** the deck opens by reminding students that Week 8 gave the CPU a way to talk to slow outside devices. Week 9 turns inward and finds the *same* speed-mismatch problem living inside the machine, between the CPU and its own DRAM. If students look lost at the opening, run the Week 8 summary frame first — the parallel (fast requester, slow responder, buffering in between) is the hook for the entire lecture.
- **Timing.** This is a 65-slide deck across six acts. Acts 4 (mapping) and 5 (replacement) are the GATE-load-bearing ones and together deserve more than half the available time. If you are running short, compress Act 2 (ROM family — it is a table students can read) rather than trimming the worked address traces.

9.2 Topic 1: Why the Hierarchy Exists

Start with the constraint, not the solution. A memory designer wants three things at once — **fast access**, **large capacity**, and **low cost per bit** — and the underlying technology refuses to give all three. The relationships run strictly against you: faster access time means higher cost per bit; greater capacity means lower cost per bit; and greater capacity means slower access time. Pick any two and the third collapses. A memory built entirely from the fastest available technology would be small and ruinously expensive; a memory built entirely from the cheapest would be enormous and unusably slow.

The escape is not a better memory technology but a better *organisation*: build a hierarchy of several memories with different technologies, and arrange for the fast ones to hold what is actually being used. This works only because of an empirical property of real programs.

Definition (Locality of reference). *Over any short interval, a program's memory references cluster into a small portion of its address space rather than scattering uniformly.* **Temporal locality:** *a location referenced now is likely to be referenced again soon (loop counters, the top of the stack, a hot variable).* **Spatial locality:** *locations near one just referenced are likely to be referenced soon (sequential instruction fetch, array traversal, struct fields).*

Note the division of labour, because students routinely conflate the two: temporal locality is what justifies *keeping* a block after you have used it, and spatial locality is what justifies *fetching a whole block* rather than a single word. Every design decision later in the lecture serves one or the other.

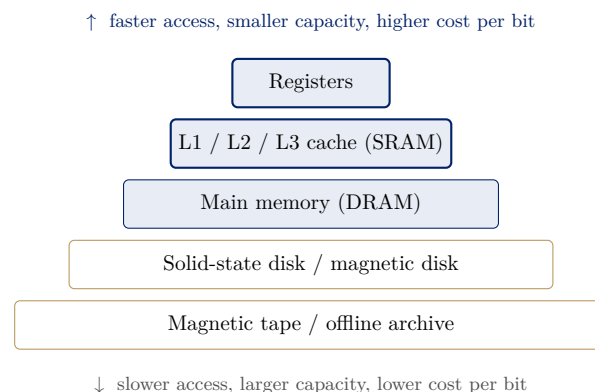


Figure 9.1: The five-tier memory hierarchy. Each level acts as a cache for the level below it (Stallings 11e, Fig. 4.6, PDF p. 160).

How to present Figure 9.1. Build it from the *bottom up*, not the top down — the coordinate map above lists the bars top-down, but the teaching order is reversed. Draw the widest bar first at the bottom (tape/archive) and say “this is everything the institution has ever stored”. Then narrow upward one bar at a time, naming what each level holds: disk holds everything this machine has; DRAM holds what this *program* needs; cache holds what this *loop* needs; registers hold what this *instruction* needs. Only after all five bars are on the board add the two arrows — the “faster, smaller, dearer” arrow above the apex and the “slower, bigger, cheaper” arrow below the base. Finish by circling the single boundary between cache and main memory and saying: *that one interface is the entire rest of today’s lecture*. That circle is what stops students expecting you to cover disks and virtual memory today (those are Week 10).

Teaching note: The recurring misconception here is that the hierarchy is a *storage* arrangement — students picture data being permanently filed at one level. It is a *replication* arrangement: the same bytes exist at several levels at once, and a level higher up holds a *copy* of something that also lives below. Ask directly: “if a value is in the cache, is it also in main memory?” Under a write-through policy, always yes. That question also plants the seed for write policy, which the deck deliberately defers.

Quantify the payoff immediately, or the hierarchy stays an abstraction. Model just two adjacent levels: level 1 (cache) with access time T_1 and **hit ratio** H , and level 2 (main memory) with access time T_2 . A miss must first search level 1 and only then go to level 2, so a hit costs T_1 and a miss costs $T_1 + T_2$:

$$T_{\text{avg}} = H T_1 + (1 - H)(T_1 + T_2) = T_1 + (1 - H) T_2.$$

Stress what H is *not*: it is not a knob the designer sets. It is an *outcome* of cache size, block size, mapping scheme, and replacement policy — which is precisely the agenda for the remaining five topics.

Board: (*Example 1 of 2 for this topic — the forward direction.*) Put $T_1 = 2$ ns and $T_2 = 80$ ns on the board and compute T_{avg} at three hit ratios. *On the board, in order:* first write the formula $T_{\text{avg}} = T_1 + (1 - H)T_2$ alone on one line; second, a three-row table with columns H / $(1 - H)$ / T_{avg} , filling the rows one at a time and speaking each substitution aloud; label the “saving” column last, after all three rows exist, so the non-linearity lands as a punchline rather than a prediction.

- $H = 0.60$: $T_{\text{avg}} = 2 + 0.40 \times 80 = 2 + 32 = 34$ ns.
- $H = 0.90$: $T_{\text{avg}} = 2 + 0.10 \times 80 = 2 + 8 = 10$ ns.
- $H = 0.99$: $T_{\text{avg}} = 2 + 0.01 \times 80 = 2 + 0.8 = 2.8$ ns.

Moving H from 0.60 to 0.90 saves 24 ns; moving it from 0.90 to 0.99 — a nine-times-smaller improvement in miss rate — saves a further 7.2 ns and brings the result to within 40% of the raw cache time. The payoff is violently non-linear, which is why real designs fight for the last percentage point.

Teaching note: Live-interaction hook. Before you compute the $H = 0.99$ row, stop and ask for a show of hands: “we went from 34 ns to 10 ns by improving the hit ratio from 0.6 to 0.9. Predict the $H = 0.99$ number — higher or lower than 5 ns?” Most students anchor on the previous *difference* and guess something near 4–6 ns; the actual 2.8 ns is closer to the floor $T_1 = 2$ ns than they expect. Reveal only after the guesses are committed, then name the reason: T_{avg} is bounded below by T_1 , so all the remaining headroom is in the $(1 - H)T_2$ term.

Board: (*Example 2 of 2 — the inverse direction, which is the form exams actually ask.*) Same $T_1 = 2$ ns and $T_2 = 80$ ns. What hit ratio is required to achieve $T_{avg} = 5$ ns? On the board, in order: write the target equation $5 = 2 + (1 - H) \times 80$; isolate $(1 - H) = 3/80$ on the next line; convert to 0.0375; only then write $H = 0.9625$ and box it. Students who can only run the formula forwards get stuck here, so make the rearrangement itself the visible step. Close by asking whether a 96.25% hit ratio is realistic — it is, comfortably, for real L1 caches, which is the reassuring answer.

9.3 Topic 2: What Main Memory Is Actually Made Of

A short detour before the cache itself, and worth keeping short. Its real purpose is to show that the SRAM/DRAM split is the hierarchy of Topic 1 repeated in miniature, inside a single machine.

Definition (SRAM and DRAM). Both are volatile (contents are lost when power is removed) and both are “random access” (every address costs the same time). **SRAM** (static RAM) stores each bit in a flip-flop of four to six transistors: no refresh needed, fast, expensive, low density — this is what cache is built from. **DRAM** (dynamic RAM) stores each bit as charge on a tiny capacitor, one transistor per cell: dense and cheap, but the charge leaks, so every cell must be periodically refreshed — this is what main memory is built from (Stallings 11e, §6.1, PDF pp. 226–236).

Two clarifications ward off the standard confusions. “Random access” means every address costs the same time; it does *not* mean the contents are random — say this explicitly, because at least one student every year believes the latter. And the SRAM/DRAM choice is the same speed-versus-density trade-off as the whole hierarchy, resolved a second time one level down.

The other family is read-only — really **read-mostly** — memory, used for boot firmware, microcontroller program store, and lookup tables. Run the family from cheapest-but-unchangeable to most-flexible-but-costliest:

Type	How it is written	How it is erased	Volatile?
ROM (mask)	At fabrication, by masking	Not possible	No
PROM	Electrically, once, by the user	Not possible	No
EPROM	Electrically	UV light, whole chip	No
EEPROM	Electrically, byte at a time	Electrically, byte-level	No
Flash	Electrically	Electrically, block-level	No

Flash sits deliberately in between EPROM and EEPROM: erasable in *blocks* rather than bytes, hence cheaper and denser than EEPROM but more flexible than EPROM. That single sentence explains why flash, and not EEPROM, is the technology behind SSDs.

Teaching note: Do not lecture this table row by row — students can read it. Instead put the four column headings on the board empty and fill the *grid* by interrogation, working down the “how it is erased” column only: “not possible, not possible, UV light, electrically byte-wise, electrically block-wise.” The progression *is* the story, and it takes ninety seconds this way versus five minutes read aloud. The misconception worth pre-empting: students often assume ROM means physically unwritable. It means non-volatile and written rarely — flash is a ROM-family member you write every day.

Board: (*Example 1 of 2 — classification under pressure.*) Name four artefacts and have the class place each in the table: (i) the BIOS/UEFI firmware of a laptop that receives vendor updates; (ii) the lookup table burned once into a mass-produced washing-machine controller; (iii) a USB thumb drive; (iv) the program store of a lab microcontroller you reflash weekly during development. On the board, in order: write the four artefacts down the left as a fresh column, take answers, then

write the technology beside each and the *reason* last. Expected answers: (i) flash — must be field-updatable; (ii) mask ROM — volume makes the mask economical and it never changes; (iii) flash — block erase, high density; (iv) EEPROM or flash — both defensible, and the defence is the point. Item (ii) is the one that trips students, because “never updated” feels like a defect until you mention the per-unit cost at a million units.

Board: (*Example 2 of 2 — the cost of refresh, as a calculation.*) DRAM’s defining overhead is worth one number rather than an adjective. Take an illustrative DRAM with 8192 rows that must all be refreshed within every 64 ms window, where refreshing one row occupies the array for 100 ns. *On the board, in order:* first the total refresh work per window, $8192 \times 100 \text{ ns} = 819.2 \text{ } \mu\text{s}$; second the window itself, $64 \text{ ms} = 64000 \text{ } \mu\text{s}$; third the ratio, $819.2/64000 = 1.28\%$; box the percentage last. So refresh steals roughly 1.3% of DRAM’s availability — small, but it is a tax SRAM simply does not pay, and it is one more reason cache is SRAM.

Teaching note: State plainly that these refresh figures are *illustrative* numbers chosen to be representative, not a quoted result from our anchor text — Stallings §6.1 describes the refresh mechanism but this particular arithmetic is ours. Modelling that distinction out loud is worth more to students than the number itself, and it is exactly the citation discipline the course claims to follow.

9.4 Topic 3: Cache Principles

Now the main event. The cache is a small, fast SRAM placed between the CPU and DRAM, and the vocabulary has to be nailed down before any mapping scheme makes sense, because three of the four terms are routinely swapped by students.

Definition (Block, line, and tag). *Main memory is conceptually divided into fixed-size **blocks** of 2^b bytes. The cache is divided into **lines** (sometimes called slots) of exactly the same size, so that one line holds exactly one block. Because there are far more blocks than lines, each line must also record which block it currently holds: that identifier is the **tag**. A line additionally carries a **valid bit** saying whether it holds anything meaningful at all.*

The single most useful thing you can say here: **block and line are the same size; “block” names the unit in memory and “line” names the container in the cache.** Write that sentence on the board and leave it up for the rest of the lecture — Acts 4 and 5 both depend on students not muddling the two.

The read path is then mechanical. The CPU issues an address. The cache checks whether the containing block is resident. On a **hit**, the word is delivered from the cache at cache speed. On a **miss**, the *entire block* — not the single word — is fetched from main memory, loaded into a line, and the word is then delivered. Fetching the whole block is spatial locality being cashed in: we are betting that the neighbouring words will be wanted shortly, and the bet is cheap because DRAM transfers a burst about as fast as a single word.

Teaching note: The misconception to hunt here is that a miss fetches the requested word. Ask “on a miss, how many bytes cross the bus?” and wait — the instinctive answer is 4 (a word). The right answer is the full block, e.g. 32 or 64 bytes. Students who keep the wrong model will later be unable to explain why block size matters at all, and will get every “effect of increasing block size” question wrong.

Before moving on, name the design knobs, because Acts 4–6 each pick one up: cache *size*; mapping function (Topic 4); replacement algorithm (Topic 5); write policy; block size; and number of caches (levels, and split instruction/data). Say explicitly that write policy and block size are

priced in Week 10 — flagging the deferral stops the “but sir, what about write-back?” interruption from derailing Act 4.

Board: (*Example 1 of 2 — capacity bookkeeping.*) A cache holds 64 KB of data with a block size of 128 bytes, over a 32-bit address space. How many lines, and how many bits of *overhead* (tag + valid) does the cache carry? *On the board, in order:* lines first, $64 \text{ KB}/128 \text{ B} = 512$ lines; then the address split for direct mapping, $\text{offset} = \log_2 128 = 7$, $\text{index} = \log_2 512 = 9$, $\text{tag} = 32 - 9 - 7 = 16$; then overhead per line, $16 + 1 = 17$ bits; total overhead last, $512 \times 17 = 8704$ bits = 1088 bytes. Close with the ratio: about 1.7% of the data size is spent on bookkeeping. That number reappears in Topic 4 when full associativity inflates it.

Board: (*Example 2 of 2 — hit/miss narration on a tiny cache.*) Take a 4-line cache, block size 16 bytes, and walk the byte addresses 0, 4, 20, 0, 36 aloud. *On the board, in order:* draw four empty line boxes; convert each address to a block number ($0 \rightarrow 0$, $4 \rightarrow 0$, $20 \rightarrow 1$, $0 \rightarrow 0$, $36 \rightarrow 2$) writing the division under each; then mark hit or miss over each; fill the line boxes last. Result: miss, **hit**, miss, **hit**, miss. The teaching point is address 4: a *different address* from 0 but the *same block*, so it hits without any new fetch. That is spatial locality paying off in a single visible step, and it is far more convincing than the assertion.

9.5 Topic 4: Mapping Functions

This is the first of the two GATE-load-bearing acts, and it deserves the largest single share of the class. Everything rests on one question, so pose it before any scheme is named: *given an address, which line or lines may hold it, and how do we know whether the block sitting there is the one we want?* Placement and identification — the three mapping schemes are three different answers to exactly those two questions.

Definition (The master framework). A N -bit address is split into at most three fields. The **word (offset)** field, $b = \log_2(\text{block size})$ bits, selects a byte within the block and is fixed by block size alone — it never moves. The **index** field selects the line or set. The **tag** field is whatever remains: $\text{tag} = N - \text{index} - b$. The three fields always re-sum to N .

State the unifying observation early, because it converts three memorised schemes into one formula: **the index field shrinks from $\log_2 L$ bits, through $\log_2(L/k)$, to zero, and every bit the index loses, the tag gains.** Direct mapping and full associativity are not separate ideas; they are the two end-stops of one dial.

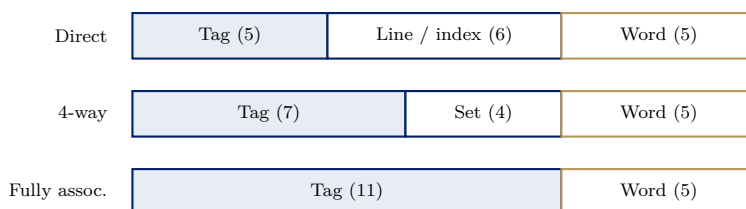


Figure 9.2: One 16-bit address, split three ways (System A: 64 KB memory, 2 KB cache, 32-byte blocks). The word field never moves; the index and tag share a fixed budget of 11 bits.

How to present Figure 9.2. The coordinate map fixes the word-field boundary at $x = 6.6$ across all three rows, and that alignment *is* the teaching point — so draw it in an order that makes the alignment visible rather than incidental. *Draw the three word fields first*, one under the other, as three identical right-hand boxes, and say: “block size is 32 bytes in all three schemes, so this field is 5 bits in all three and it is the same 5 bits.” Only then draw the internal divider in

each row — at the two-thirds point for direct, further right for 4-way, and absent entirely for fully associative. Label the tag widths (5, 7, 11) *last*, all three at once, so students see the tag growing as the divider slides right. If you draw row-by-row instead, the alignment reads as a coincidence and the punchline is lost.

Direct mapping

Block B may occupy *exactly one* line: $\text{line} = B \bmod L$ and $\text{tag} = \lfloor B/L \rfloor$, where L is the number of lines. The line number is read straight out of the address — no search, no choice (Stallings 11e, Fig. 5.7, PDF p. 190). Blocks B , $B + L$, $B + 2L$, ... all land on the same line and are distinguished *only* by the tag.

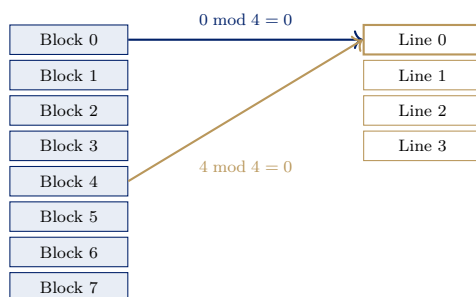


Figure 9.3: Many-to-one collision under direct mapping, drawn for a toy cache of $L = 4$ lines. Blocks 0, 4, 8, 12, ... are all forced onto line 0; only one may be resident, and the tag records which.

How to present Figure 9.3. Draw the eight block boxes down the left first, numbered, then the *four* line boxes on the right — and pause on the asymmetry before any arrow exists: eight candidates, four containers. Then draw only *two* arrows, block 0 and block 4, both landing on line 0, and write the modulo arithmetic beside each. Resist drawing all eight arrows; the deck’s diagram deliberately omits them because eight crossing arrows teach nothing that two do not. Finish by asking what happens if the program alternates between block 0 and block 4 — and let the class say “they keep evicting each other” rather than saying it yourself. That is the conflict-miss definition, discovered instead of announced.

Teaching note: The misconception that costs marks: students conclude from this figure that direct mapping is simply bad. Correct it on the spot — direct mapping needs *one* comparator of *five* bits, so it is the fastest and cheapest scheme by a wide margin, and its weakness is narrow and specific: two hot blocks that happen to be exactly L apart. On workloads without that pathology it is excellent. Exams reward the trade-off answer, never the “associativity is better” answer.

Board: (*Example 1 of 2 for mapping — a fresh system, direct mapped.*) Do *not* reuse the deck’s System A; students already have that trace. Introduce **System B**: main memory 1 MB, cache 16 KB, block size 64 bytes, probe address 0xA5C7C. *On the board, in order:* (i) address width, $1 \text{ MB} = 2^{20} \Rightarrow N = 20$ bits; (ii) word field, $64 = 2^6 \Rightarrow b = 6$; (iii) lines, $L = 2^{14}/2^6 = 2^8 = 256 \Rightarrow \text{index} = 8$; (iv) tag = $20 - 8 - 6 = 6$; (v) the check $6 + 8 + 6 = 20$ ✓ — write the check every single time, and say that a student who writes it will catch most of their own errors. Only then slice the address. Write 0xA5C7C in binary as 10100101110001111100 and cut it 6 | 8 | 6:

- Tag = $101001_2 = 41$, Index = $01110001_2 = 113$, Word = $111100_2 = 60$.

Then verify by the *other* route, arithmetic rather than bit-slicing, exactly as the deck insists: $0xA5C7C = 679036$; block $B = \lfloor 679036/64 \rfloor = 10609$; offset $= 679036 - 10609 \times 64 = 60$ ✓; line $= 10609 \bmod 256 = 113$ ✓; tag $= \lfloor 10609/256 \rfloor = 41$ ✓. Two independent routes, same answer — that is the exam technique you are actually teaching.

Fully associative and k -way set associative

Fully associative mapping removes the index entirely: a block may occupy *any* line, so the tag must be the whole block number ($N-b$ bits) and every line’s tag must be compared in parallel. Maximum placement freedom, maximum identification cost. k -way set associative is the compromise: group the L lines into $S = L/k$ **sets**; the index selects one set, and the block may occupy any of the k ways within it, so k comparators suffice.

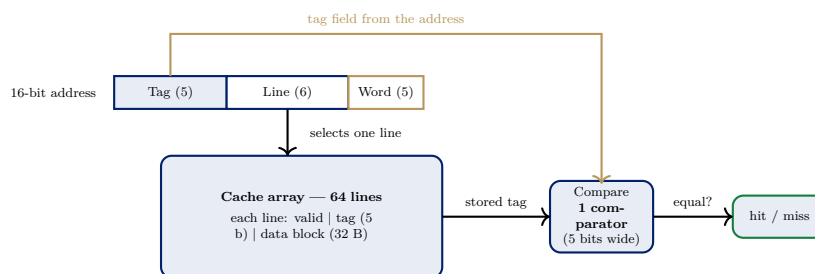


Figure 9.4: Direct-mapped hardware. The line field is a *decoder input*, not something searched for; only after a line is selected does a single comparison decide hit or miss.

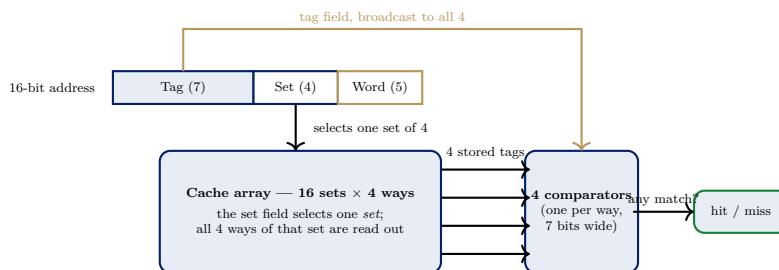


Figure 9.5: 4-way set-associative hardware. Identical skeleton to Figure 9.4: one comparator becomes k , and two bits move from the index field into the tag.

How to present Figures 9.4 and 9.5 — as one drawing, twice. These two figures share a skeleton deliberately, and the pedagogy only works if you exploit that. *Draw Figure 9.4 once and do not erase it.* Its build order, from the coordinate map: the address strip along the top first; then the big cache-array box below it; then the vertical arrow down from the *index* field into the array, narrating “the index is a decoder input — it *selects*, it does not search”; then the comparator box to the right; then the long gold path carrying the *tag* field over the top of the array and down into the comparator — draw that one last and slowly, because the whole architecture is in it: index goes down into the array, tag goes across into the comparator. Finish with the hit/miss box.

Then, for Figure 9.5, *modify rather than redraw*. Change three things in front of the class: slide the tag/index divider two bits to the right (5 | 6 becomes 7 | 4); relabel the array “16 sets $\times$ 4 ways”; and replace the single comparator with a stack of four, adding the four horizontal tag lines. Everything else stays. Say explicitly: “associativity did not change the architecture, only the

width of two fields and the number of comparators.” Students who see the two diagrams drawn independently memorise two pictures; students who watch one morph into the other understand one dial.

Teaching note: Live-interaction hook, placed here. Before you add the fourth comparator, ask: “comparators cost area and sit on the critical path. If 4-way needs 4, and fully associative on this 64-line cache needs 64 — what does the tag storage do over the same range?” Take guesses. Most students expect tag storage to blow up proportionally too. The answer is that it grows by only *one bit per line* per doubling of k : 320 bits at direct, 448 at 4-way, 704 fully associative — a mere 2.2 $\times$, against a 64 $\times$ explosion in comparators. **It is the comparators, not the tag bits, that make full associativity unaffordable.** That inversion of expectation is the single most memorable thing in Act 4.

Board: (*Example 2 of 2 for mapping — System B again, now 8-way.*) Keep System B on the board and change only the associativity: 1 MB memory, 16 KB cache, 64-byte blocks, now **8-way set associative**, same probe address 0xA5C7C. *On the board, in order:* (i) note what is *unchanged* — $N = 20$, $b = 6$, $L = 256$ — and say aloud that neither address width nor block size nor line count depends on associativity; (ii) sets, $S = L/k = 256/8 = 32 \Rightarrow$ index = 5 bits; (iii) tag = $20 - 5 - 6 = 9$ bits; (iv) the check, $9 + 5 + 6 = 20$ ✓. Then slice the same binary string 1010 0101 1100 0111 1100 as 9 | 5 | 6: tag = $101001011_2 = 331$, set = $10001_2 = 17$, word = $111100_2 = 60$. Cross-check arithmetically from $B = 10609$: set = $10609 \bmod 32 = 17$ ✓, tag = $\lfloor 10609/32 \rfloor = 331$ ✓. Finally point at the two results side by side: going from direct ($k = 1$) to 8-way, the index lost exactly 3 bits and the tag gained exactly 3 — and the word field never moved. **Doubling k always trades one index bit for one tag bit.** If students take one transferable rule from Act 4, make it that one.

9.6 Topic 5: Replacement Algorithms

The second GATE-load-bearing act. Open by narrowing the question sharply, because students over-apply replacement: *under direct mapping there is no decision to make at all* — the incoming block has exactly one legal line, and whatever is there is evicted. A replacement policy is needed only when the block has a *choice* of homes: fully associative (choose among all L lines) or k -way set associative (choose among the k ways of one set). Getting students to say that back to you is worth the thirty seconds.

The five policies, in the order the deck presents them:

- **FIFO** — evict the block that has been resident longest, regardless of use. Tracks load order only.
- **LRU** — evict the block unused for the longest time. Tracks recency.
- **LFU** — evict the block with the smallest reference count. Tracks frequency.
- **Random** — evict an arbitrary block. Tracks nothing.
- **OPT** (Belady’s MIN) — evict the block whose *next* use is furthest in the future. Requires the future; unimplementable, and used only as a benchmark.

Teaching note: Sourcing, stated out loud. Stallings 11e covers LRU, FIFO, LFU, and Random only. OPT and Belady’s anomaly are standard general treatment carried in no page

of our anchor text — present them as such, and if pressed, point students to an operating-systems text’s page-replacement chapter. Modelling this honesty is part of the lecture, not an interruption to it.

The deck runs all five against one common reference string (7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 on a 3-line fully associative cache) and gets 3, 4, 5, 3, and 6 hits respectively — FIFO, LRU, LFU, Random, OPT. Walk at least FIFO and LRU live; the others can be shown as completed tables. The headline is the spread: identical input, identical cache, and a **2× range in hit ratio** from 23.1% to 46.2%. The policy is not a detail.

Teaching note: The misconception that matters most in this act: that LRU is simply the best implementable policy. On the deck’s string LFU beats it (5 hits versus 4), and the board example below shows a string where even *FIFO* beats LRU outright. Say the honest version: no implementable policy dominates on all strings; LRU is preferred in practice because it is robust across *typical* workloads and cheap at low associativity, not because it wins every trace. Students who learn “LRU is best” will confidently mis-answer any exam question engineered around a counterexample — and examiners engineer them precisely because the misconception is so common.

Board: (*Example 1 of 2 for replacement — a fresh string where FIFO beats LRU.*) Use a 3-line fully associative cache, initially empty, and the string

2, 3, 4, 2, 5, 3, 4, 5, 2, 3 (10 references).

On the board, in order: draw the reference row across the top first; then three empty slot rows beneath it; then a hit/miss row at the bottom — the empty grid before any content, so students can copy the frame while you fill it. Fill *column by column*, never row by row, narrating the eviction decision at each miss. Run FIFO first:

- References 2, 3, 4: three compulsory misses; queue order is [2, 3, 4].
- Reference 2: **hit** (FIFO does *not* reorder on a hit — say this aloud, it is the whole difference from LRU).
- Reference 5: miss; evict the queue head 2; residents {3, 4, 5}, queue [3, 4, 5].
- References 3, 4, 5: three consecutive **hits**.
- Reference 2: miss; evict head 3; residents {4, 5, 2}.
- Reference 3: miss; evict head 4.
- **FIFO result: 4 hits, 6 misses.**

Now re-run the identical string under LRU on a fresh grid: references 2, 3, 4 miss; reference 2 **hits** and promotes 2 to *most-recent*; reference 5 misses and evicts 3 (least recently used); reference 3 misses and evicts 4; reference 4 misses and evicts 2; reference 5 **hits**; reference 2 misses and evicts 3; reference 3 misses and evicts 4. **LRU result: 2 hits, 8 misses** — half of FIFO’s.

Teaching note: Live-interaction hook — a prediction trap, and the best one in the lecture. Before running the LRU pass, tell the class you are about to run the same string under LRU and ask them to commit, by show of hands, to whether LRU will do better, worse, or the same as FIFO’s 4 hits. Nearly everyone votes “better”, because the deck’s own common string had LRU beating FIFO. The answer is *worse* — LRU manages only 2. Let the surprise

sit before explaining: reference 2 at step 4 promotes block 2 to most-recent under LRU, which pushes block 3 to the bottom and gets it evicted at step 5 — just before 3 is needed at step 6. FIFO, by ignoring the hit entirely, happens to keep the block LRU throws away. **Recency is a heuristic about the future, and heuristics can be wrong.**

Board: (*Example 2 of 2 — OPT on the same string, to price the heuristics.*) Keep the same 3-line cache and the same string 2, 3, 4, 2, 5, 3, 4, 5, 2, 3, and run OPT. *On the board, in order:* at each miss, write the residents' *next-use positions* in a small side column *before* naming the victim — that side column is the entire method, and students who skip it guess. References 2, 3, 4 miss; reference 2 **hits**; at reference 5 the residents are {2, 3, 4} with next uses at steps 9, 6, and 7 respectively, so block 2 (furthest, step 9) is evicted; references 3, 4, 5 then all **hit**; at reference 2 the residents are {5, 3, 4} and both 5 and 4 are never used again, so either may go; reference 3 **hits**. **OPT result: 5 hits, 5 misses.** Line the three up on the board — OPT 5, FIFO 4, LRU 2 — and close the act: OPT is the ceiling no implementable policy can beat, FIFO got unusually lucky here, LRU got unlucky, and *one string proves nothing about policies in general — it proves only that policies diverge*, which is why real designs are chosen by simulation over real traces rather than by argument.

Belady's anomaly

The claim that ought to be impossible: under **FIFO**, *increasing* the number of cache lines can *increase* the number of misses on the same string. The deck demonstrates it on the string

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 :

with 3 lines, 9 misses; with 4 lines, 10 misses.

How to present it. Draw both grids *side by side on the same board*, three slots on the left and four on the right, and fill them in lockstep column by column — reference 1 on both, then reference 2 on both, and so on. The anomaly is invisible if you complete one table and then the other, because by then students have lost the first. Filled in parallel, the divergence is watchable: the 4-line cache is comfortably ahead early (it **hits** at references 5 and 6 where the 3-line cache misses) and then loses that lead and more over the tail. Write the two totals at the bottom simultaneously and let the class read them off.

Then give the explanation, which is the part exams reward: **LRU and OPT are immune** because they are *stack algorithms* — the set of blocks held with n lines is always a subset of the set held with $n + 1$ lines, so extra capacity can never remove a block that would otherwise have hit. FIFO has no such property: adding a line changes the eviction *order* itself, not merely its depth.

Close the act with the hardware cost of each policy, since it explains why the theoretically-best policy is rarely built: LRU on a *two-way* set needs a single USE bit per set; LRU on a fully associative cache needs an L -entry recency list reordered on every access — which is why exact LRU is rare beyond small associativities. FIFO needs a $\lceil \log_2 k \rceil$ -bit round-robin pointer per set with no update at all on a hit; LFU needs a counter per line plus a comparison tree, and periodic ageing to stop stale counts dominating; Random needs no per-line state whatsoever. **Cost rises with how much history the policy remembers** — Random remembers nothing, exact LRU remembers everything.

9.7 Topic 6: Cache Performance

The final act puts numbers on everything. Define the terms once, precisely: for n references of which n_h hit, $H = n_h/n$ and the miss ratio is $1 - H$, and

$$\text{AMAT} = T_{\text{hit}} + (1 - H) \times T_{\text{miss penalty}}.$$

Reconcile this with Topic 1's two-level form explicitly, because students think they are different formulas: $T_{\text{avg}} = HT_1 + (1 - H)(T_1 + T_2)$ expands to $T_1 + (1 - H)T_2$, which *is* AMAT with $T_{\text{hit}} = T_1$ and miss penalty $= T_2$. One formula, two spellings.

Teaching note: The single most common arithmetic error in this act: treating the miss penalty as the total miss time and adding the hit time twice. A miss penalty is the *extra* time *beyond* the hit time, because the hit time is paid on every access, hit or miss. Give students the sanity test and insist on it: **any AMAT answer must lie strictly between the hit time and the full miss time.** An answer outside that interval is wrong without further checking — and it costs nothing to verify.

Board: (*Example 1 of 2 — AMAT from a raw reference count.*) A program makes 5000 memory references, of which 4600 hit. Cache access time is 3 ns; a miss costs a further 120 ns to fetch the block. *On the board, in order:* hit ratio first, $H = 4600/5000 = 0.92$, and miss ratio 0.08 beside it; then $\text{AMAT} = 3 + 0.08 \times 120 = 3 + 9.6 = 12.6$ ns; then the cross-check in the other form, $0.92 \times 3 + 0.08 \times 123 = 2.76 + 9.84 = 12.6$ ns ✓; then the sanity test last, $3 < 12.6 < 123$ ✓. Doing the cross-check every time is what stops the double-counting error from ever appearing in a script.

Then the two-level case, where a second trap lives. With an L1 (time t_1 , hit ratio h_1), an L2 (time t_2 , *local* hit ratio h_2 measured over L1 misses only), and main memory (time t_m), two access models decide the arithmetic:

$$\textbf{Hierarchical: } T = h_1 t_1 + (1 - h_1) h_2 (t_1 + t_2) + (1 - h_1)(1 - h_2)(t_1 + t_2 + t_m),$$

$$\textbf{Simultaneous: } T = h_1 t_1 + (1 - h_1) h_2 t_2 + (1 - h_1)(1 - h_2) t_m.$$

Tell students to read every question for which model is intended, and that absent any statement they should assume hierarchical — it is what real multi-level caches do.

Board: (*Example 2 of 2 — both models, and the local/global trap.*) Take $t_1 = 2$ ns with $h_1 = 0.90$; $t_2 = 15$ ns with *local* $h_2 = 0.80$; $t_m = 150$ ns. *On the board, in order:* draw the three-case probability tree first — L1 hit / L1 miss & L2 hit / both miss — and write the three probabilities 0.90, 0.08, 0.02 on its branches *before* any time appears; then attach the hierarchical times to each branch; then sum; and write the weight check last.

- L1 hit: $0.90 \times 2 = 1.8$ ns.
- L1 miss, L2 hit: $0.10 \times 0.80 \times (2 + 15) = 0.08 \times 17 = 1.36$ ns.
- Both miss: $0.10 \times 0.20 \times (2 + 15 + 150) = 0.02 \times 167 = 3.34$ ns.
- **Hierarchical total:** $T = 1.8 + 1.36 + 3.34 = 6.5$ ns. Weight check: $0.90 + 0.08 + 0.02 = 1.000$ ✓.
- Simultaneous, same numbers: $0.90(2) + 0.08(15) + 0.02(150) = 1.8 + 1.2 + 3.0 = 6.0$ ns.

Then spring the trap explicitly, because it is a full-marks error rather than a rounding one: here $h_2 = 0.80$ is *local* — 80% of the references that *reach* L2. The **global** L2 hit ratio is $(1 - h_1)h_2 = 0.10 \times 0.80 = 0.08$, i.e. only 8% of all references. Teach students the linguistic tell: the phrase “of the accesses that miss in L1” marks a *local* ratio; “of all memory references” marks a *global* one. If a problem states neither, say which you assumed.

Teaching note: Live-interaction hook to close the lecture. Pose the budget question and take a vote before revealing anything. Starting from $t_1 = 1$ ns, $h_1 = 0.95$, $t_m = 100$ ns and no L2 — so $\text{AMAT} = 1 + 0.05 \times 100 = 6$ ns — you may spend your entire budget on exactly one of: (a) halving the hit time to 0.5 ns, or (b) raising the hit ratio to 0.975. Most classes vote (a), because halving *feels* like the bigger intervention. The arithmetic: (a) gives $0.5 + 0.05 \times 100 = 5.5$ ns, a 0.5 ns saving; (b) gives $1 + 0.025 \times 100 = 3.5$ ns, a 2.5 ns saving — **five times better**. The lesson, and the last thing to leave on the board: when the miss penalty is large, the miss *rate* dominates AMAT and the hit time is a second-order term. That is precisely why the lecture spent its two longest acts on mapping and replacement — both are miss-rate levers.

9.8 Summary — One-Page Pre-Class Refresh

- **Why a hierarchy:** fast / large / cheap cannot be had together; locality (temporal $\rightarrow$ keep a block, spatial $\rightarrow$ fetch a whole block) makes a layered organisation work. Two-level model: $T_{\text{avg}} = T_1 + (1 - H)T_2$. H is an outcome, not a knob.
- **Main memory:** SRAM (flip-flop, no refresh, fast, dear $\rightarrow$ cache) versus DRAM (capacitor, refresh needed, dense, cheap $\rightarrow$ main memory); both volatile. ROM family in order of flexibility: mask ROM, PROM, EPROM, EEPROM, flash (block erase — hence SSDs).
- **Cache vocabulary:** block (unit in memory) = line (container in cache) in size; tag identifies which block a line holds; valid bit says whether it holds anything. A miss fetches the *whole block*.
- **Master framework:** tag = $N - \text{index} - b$, and the three fields always re-sum to N . Word field is fixed by block size and never moves. Doubling k trades one index bit for one tag bit.
- **Mapping:** direct (line = $B \bmod L$, tag = $\lfloor B/L \rfloor$, 1 comparator, conflict misses); fully associative (no index, tag = block number, L comparators, needs replacement); k -way (index selects one of $S = L/k$ sets, k comparators). System A (64 KB / 2 KB / 32 B, address 0x3F8C): direct $\rightarrow$ tag 7, line 60, word 12; 4-way $\rightarrow$ tag 31, set 12; fully $\rightarrow$ tag 508. Cost: comparators double with k , tag storage grows only one bit per line — comparators are the binding constraint.
- **Replacement** (needed only where a block has a choice of homes): FIFO, LRU, LFU, Random, OPT. On the deck’s string: 3/4/5/3/6 hits respectively. No implementable policy dominates on all strings. Belady’s anomaly — more FIFO lines can mean more misses (9 at 3 lines, 10 at 4); LRU and OPT are immune because they are stack algorithms. Hardware cost rises with remembered history: Random (nothing) $\rightarrow$ FIFO (pointer) $\rightarrow$ LRU (USE bit at 2-way; full list otherwise) $\rightarrow$ LFU (counters + ageing).
- **Performance:** $\text{AMAT} = T_{\text{hit}} + (1 - H)T_{\text{miss penalty}}$; miss penalty is the *extra* time; answer must lie between hit time and full miss time. Two-level: hierarchical (times add) versus

simultaneous (only the answering level counts); assume hierarchical unless told otherwise. Local versus global L2 hit ratio is a full-marks distinction. Miss rate beats hit time when the penalty is large.

- **Citation discipline to hold at the board:** Stallings 11e Ch. 4 / Ch. 5 §§5.1–5.2 / Ch. 6 §6.1 are page-citable; P&H §5.3 (p. 397) for AMAT; Hamacher Ch. 8 chapter-level only; **OPT and Belady’s anomaly get no page citation at all** — they are not in our anchor text.
- **Bridge to Week 10:** the same block-placement question, asked one level down — pages instead of blocks, main memory instead of cache, plus associative memory, segmentation, and secondary storage.